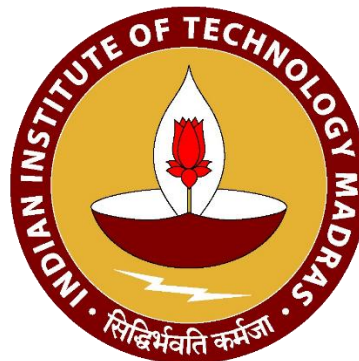


Python for Data Science

Web M. Tech Course DA5301W

Module 4: Git Fundamentals

Dr. P.S Jayadev



Contents

- Motivation for Git
- What is Git?
- Places where Code Lives in Git
- Git Workflow and Terminology
- Branching and Merging in Git
- Collaboration via Git

Motivation for Git

- Coding projects often evolve gradually, continuously, and collaboratively.
- Common challenges include:
 - Multiple versions of the same file scattered across systems
 - Uncertainty about which version has the working or tested features
 - Difficulty in collaboration, as team members must manually track and merge changes
- Data science projects also evolve iteratively
- Git provides a structured way to manage this evolution

What is Git?

- Git is a distributed version control system
- A tool that helps teams track, manage, and collaborate on code efficiently
- Originally developed by Linus Torvalds (creator of Linux) in 2005
- **Key Characteristics**
 - Every developer has a complete copy of the project history
 - Works offline, syncing with others' work when connected
 - Designed for speed, flexibility, and reliability
 - Widely adopted across the industry
- **What it does**
 - Maintains a complete version history of every saved change in a repository
 - Enables safe experimentation through branching and merging
 - Makes collaboration organized and transparent
 - Open source and supported universally (Any OS and Any language)

Git Workflow

Places Where Code Lives in Git

- Every project in Git has **four places** where a file can exist

Working Directory

Your project folder — where you write, test, and modify your code files

Files here are not tracked yet for changes

Staging Directory

A preparation zone between working dir and local repo

Here you select which changes to include in the next version

Saved as a snapshot

Local Repository

Maintains a machine level copy of all changes made and recorded in the project

Changes or snapshots from staging are committed (saved) here

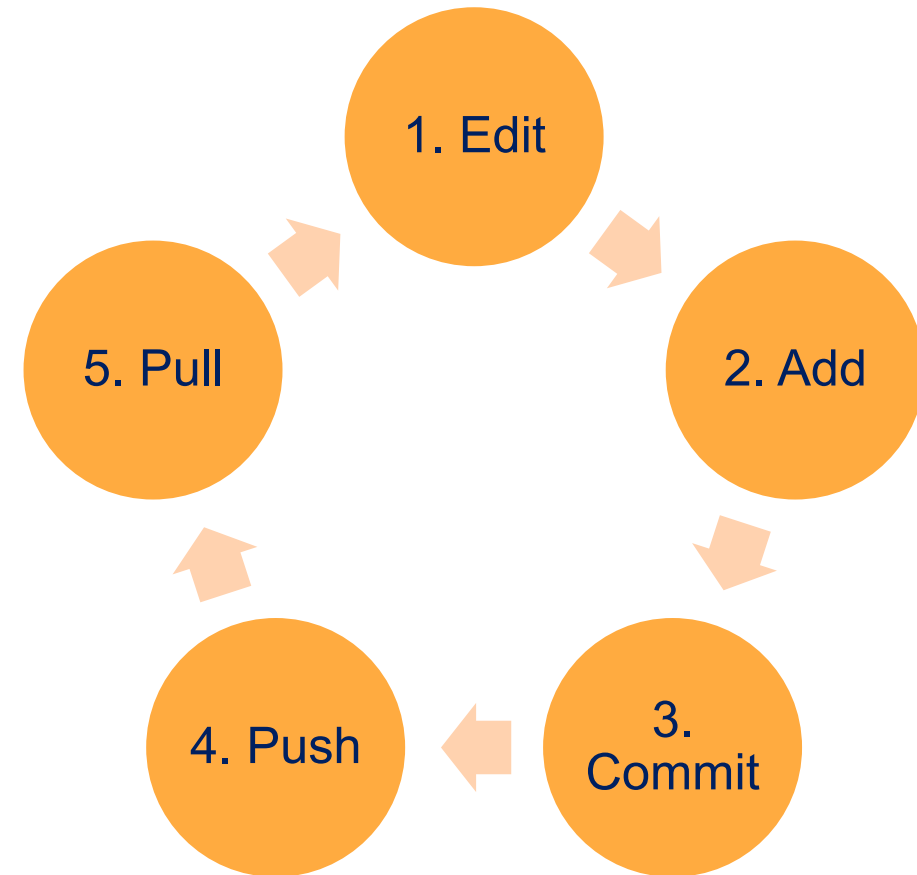
Remote Repository

Hosted on platforms like github, bitbucket for universal accessibility

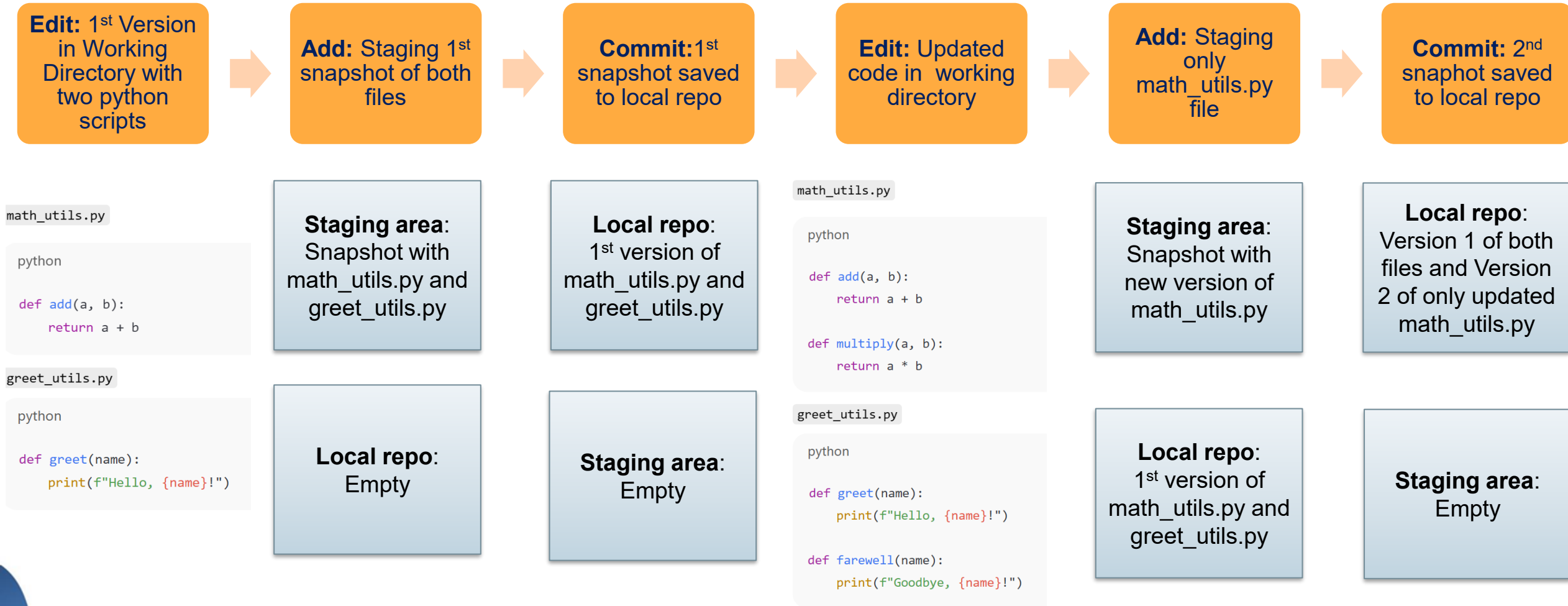
Enables team collaboration, review and backup

Git Workflow and Key Terminology

- Git follows a simple yet powerful cycle
- Each step moves the code through the 4 places where it lives
- **Git Workflow:**
 - **Edit:** Modify your files in the working directory
 - **Add:** Select which files/changes to include in the next version and prepares a snapshot in staging area
 - **Commit:** Save a version of the staged changes to local repository
 - **Push:** Share your commits with the remote repository
 - **Pull:** Fetch and merge updates from others from remote to local repository

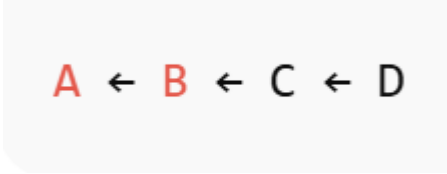


Git Workflow : Simple Example



How Git Handles Commits

- Each commit in Git has a unique hash ID that acts like a permanent reference to its snapshot
- Each commit stores the following:
 - The snapshot of files at that point
 - A pointer to its parent commit
 - Metadata (author, message, timestamp, etc.)
- In other words, commits in git form a linked chain
- Therefore, git contains a graph of directed history of commits



A ← B ← C ← D

The diagram illustrates a linked chain of commits. It shows four labels: A, B, C, and D, arranged horizontally from left to right. Between each label is a left-pointing arrow (←). The labels A and B are colored red, while C and D are black. This visualizes the concept that each commit points to its parent commit, forming a directed graph of the commit history.

Branching and Merging in Git

Branching in Git

- Motivation:

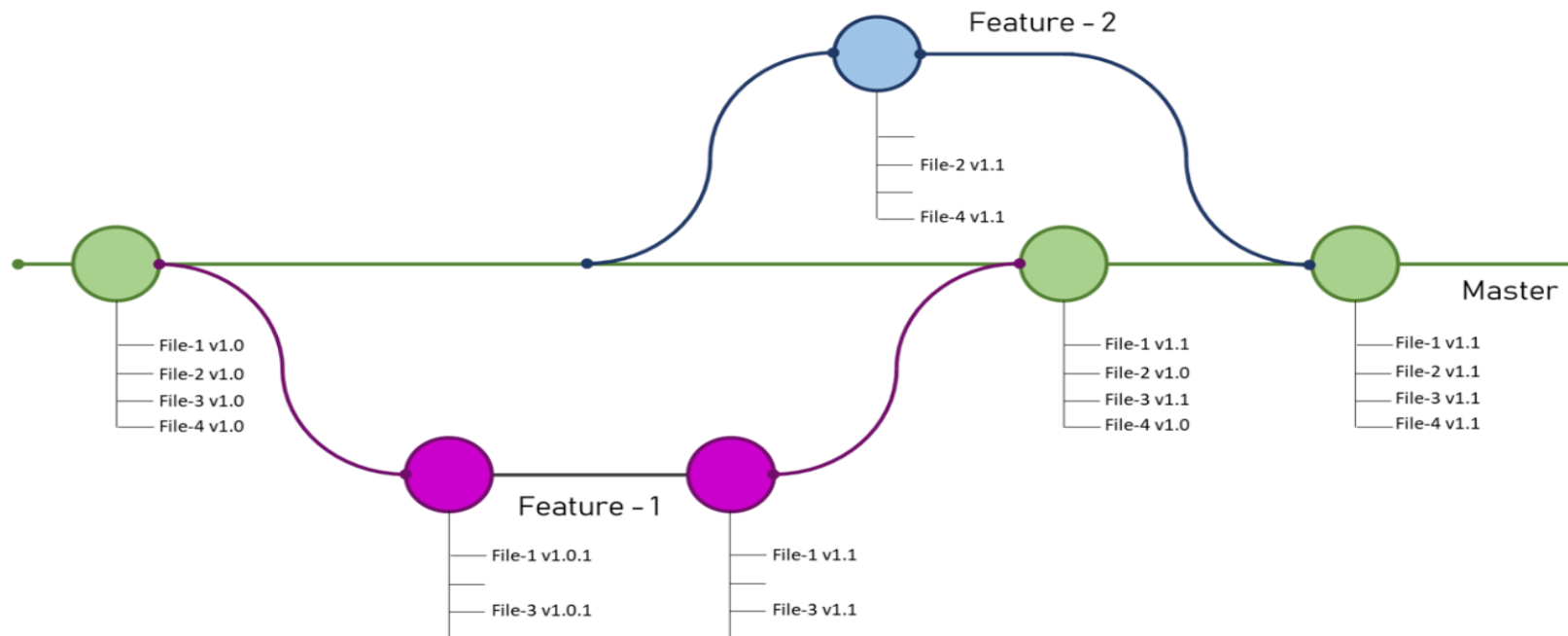
- Software and data-science projects evolve through experimentation
- You need to try a new idea without disturbing the working version

- Branch:

- Branching refers to creating a separate timeline of your project
- Each branch is given a unique name for identification
- New features can be developed, bugs can be fixed in isolation
- Original branch (main or master) stays stable while new branches explore different directions
- Branches can be merged when needed

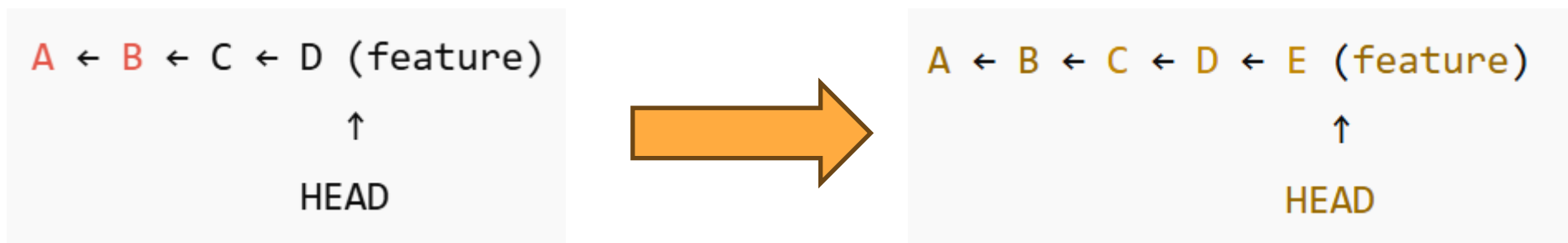
Advantages of Branching in Git

- Encourages safe experimentation — try new code or analyses freely.
- Enables parallel development — multiple teammates can work simultaneously.
- Makes feature isolation easy — bugs in one branch don't affect others.
- Simplifies collaboration and review



How Git Handles Branches

- A branch in Git is simply a movable pointer (label) that points to a specific commit
- Branch name (like main, feature-1) is just an alias for the latest commit in that timeline
- Branch pointer moves as one makes a new commit on the branch
- **HEAD** is a special pointer that tells Git where you currently are i.e., latest commit in the current branch
- This makes branching and merging extremely fast



Local and Remote Branches

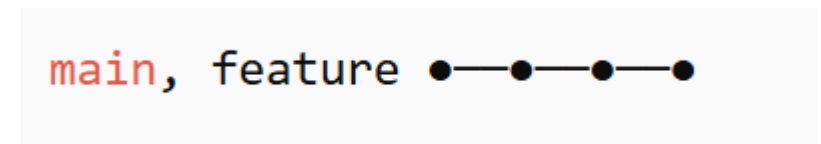
- Branches can exist locally on your machine or remotely on shared platforms like GitHub
- Both can be connected, synchronized, or kept separate depending on your workflow
- **Local branch:**
 - Created in your local repository
 - Exists only on your system until you share it remotely
 - Acts like a personal workspace for experimentation
 - Can be pushed to the remote repository when ready
- **Remote Branch:**
 - Lives in the remote repository
 - Accessible to all collaborators
 - Can be pulled or fetched into your local repo to continue work
 - Often used for team collaboration

Merging after Branching

- Merging is combining two versions of the code into one consistent codebase
- What happens during merging:
 - Git looks at both branches
 - It compares what changed in each line code since they split
 - Then it joins the differences if they don't overlap
 - A new “merged” version is created combining both branches
- Various scenarios in merging:
 - Merging a Branch with Its Ancestor
 - Merging Two Branches with a Common Ancestor
 - Merging with Conflicts

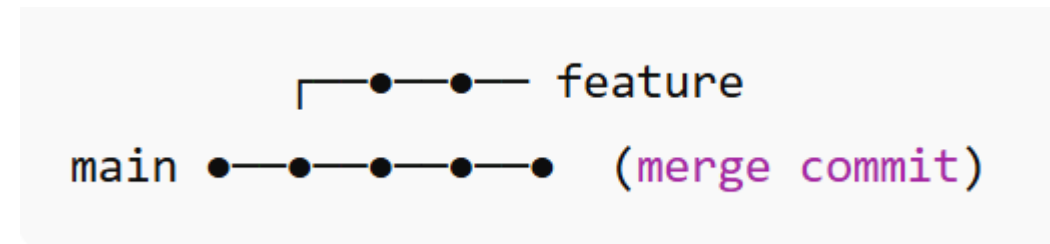
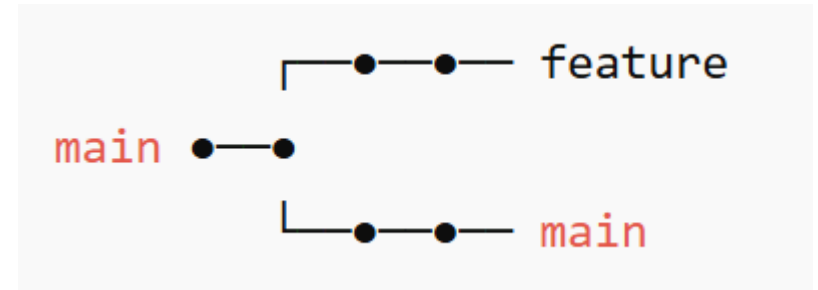
Scenario I: Merging a Branch with Its Ancestor

- New branch (say 'feature') is branched out of 'main' branch
- New commits are made on 'feature' branch and no changes are made on 'main' branch
- Now when 'feature' is to be merged back into 'main' branch
- Since there are no changes in main, git simply moves the pointer of main to latest commit in feature – **fast forward of history**
- No divergence and no conflict possible



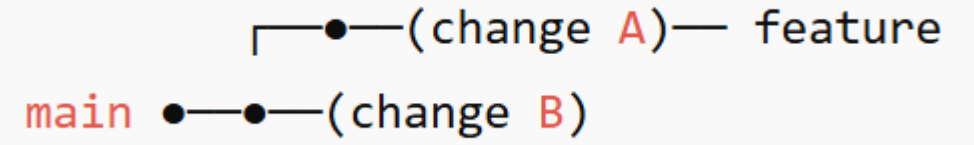
Scenario 2: Merging Two Branches with a Common Ancestor

- New branch 'feature' started from 'main'
- Both branches developed new commits independently i.e., both are different with a common ancestor
- Now when 'feature' is to be merged back into 'main' branch
- Git looks at 3 commits: last one before split, latest commit in 'main' and 'feature'
- If there are no overlapping changes, then all unique changes since split are combined
- **New merge commit** is created after three-way merge



Scenario 3: Merging with Conflicts

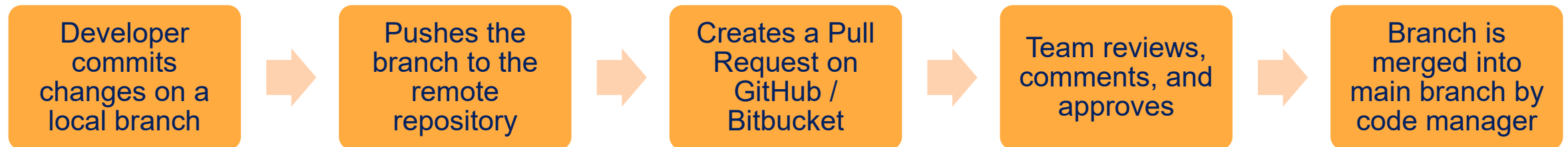
- Similar to scenario 2 but happens when both branches edited same lines of code differently
- Git cannot decide which change to keep
- Git pauses the merge and marks conflict areas for review
- One has to manually review the changes and decide the final version for merge commit



Collaboration via Git

Pull Requests (PR) for Collaboration

- Formal request to merge changes from one branch (often your feature branch) into another (usually main) in a shared remote repository
- Allows teammates to peer-review, discuss and approve work before it becomes part of the main project
- PR ensures code quality, proper collaboration and accountability
- Typical PR workflow:



Cloning and Forking

- Cloning:

- Used to create a local copy of existing remote repo
- You get the entire project history and can start contributing to the project
- Typical used by collaborators within the same team or organisation
- Cloning is possible only when has read access to the repository
- Public repositories can be cloned by anyone
- To push changes back to the remote, the user must have write permissions.

- Forking:

- Creates a personal copy of someone else's repository on GitHub or Bitbucket
- One can experiment freely without affecting the original project
- Public repositories can be forked by anyone else one needs read access
- Common in open-source collaboration
- Contributors can raise a PR to propose their changes to the original project

Basic Git Commands

Basic Git Commands

`git init`

Initialize a new Git repository in the current folder

`git clone <URL>`

Create a local copy of an existing remote repository

`git status`

Show which files are staged, unstaged, or untracked

`git add <file>`

Move changes from working directory to staging area

`git commit -m "message"`

Save staged changes to the local repository

`git log`

Gives the commit history of the current branch in the local repo

`git diff`

Compare differences between versions or stages based on parameters given

Basic Git Commands

`git branch`

List, create, or delete branches

`git checkout <branch>`

Move to another branch

`git merge <branch>`

Combine another branch into the current one

`git push`

Upload commits from local to remote repository

`git pull`

Fetch and integrate changes from remote to local

`git fetch`

Download updates from remote without merging

Reverting Code under Different Scenarios

Undo changes before Staging

- **Scenario:** You modified a file but haven't staged it yet
- **Goal:** Discard local edits and restore the last committed version
- **Command:** `git restore <file>`

Undo changes after staging but before Commit

- **Scenario:** You used `git add`, but haven't committed yet
- **Goal:** Unstage the file (keep your edits, just remove from staging area)
- **Command:** `git restore --staged <file>`

Undo the last commit (keep changes locally)

- **Scenario:** You committed too early or forgot something.
- **Goal:** Undo the commit but keep the changes in your working directory.
- **Command:** `git reset --soft HEAD~1`
- HEAD is moved to previous commit

Reverting Code under Different Scenarios

Undo the last commit completely (discard changes)

- **Scenario:** You made a wrong commit and want to erase it entirely.
- **Goal:** Remove both the commit and the changes.
- **Command:** `git reset --hard HEAD~1`

Revert a commit safely (keep history intact)

- **Scenario:** You already pushed the bad commit to the remote.
- **Goal:** Create a new commit that undoes the effects of a previous one.
- **Command:** `git revert <commit-hash>`
- Commit hash can be obtained using `git log`

Best Practices in Git Usage

Commit Discipline

- Commit often, but logical chunks of work
- Write clear commit messages
- Avoid large, unrelated commits otherwise it is difficult to revert

Branching Habits

- Create branches for new features or experiments
- Keep main or master stable and deployable
- Delete merged branches to keep repo clean

Collaboration

- Pull before you push: stay updated to avoid conflicts
- Communicate before large merges
- Review code before merging (via Pull Requests)

Conflict Prevention

- Commit frequently and sync to remote regularly
- Test merges locally for conflicts before pushing

Summary

- Git is your project's timeline — every commit is a snapshot.
- Code managed by git lives in 4 places: Working directory, staging, local and remote repos
- Git Workflow: Edit — Add — Commit — Push — Pull
- Use branches to experiment safely
- Use merging and PRs to collaborate effectively
- Follow good practices to avoid conflicts and confusion